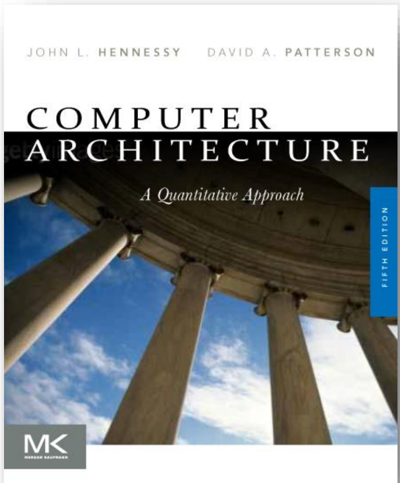

Computer Architecture

Lec 18-20
Instruction-Level Parallelism

Introduction

- Pipelining become universal technique in 1985
  - Overlaps execution of instructions
  - Exploits "Instruction Level Parallelism"
- Beyond this, there are two main approaches:
  - Hardware-based dynamic approaches
    - Used in server and desktop processors
    - Not used as extensively in PMP processors
  - Compiler-based static approaches
    - Not as successful outside of scientific applications

Instruction-Level Parallelism

- When exploiting instruction-level parallelism, goal is to maximize CPI
  - Pipeline CPI =
    - Ideal pipeline CPI +
    - Structural stalls +
    - Data hazard stalls +
    - Control stalls
- Parallelism with basic block is limited
  - Typical size of basic block = 3-6 instructions
  - Must optimize across branches

Data Dependence

- Loop-Level Parallelism
  - Unroll loop statically or dynamically
  - Use SIMD (vector processors and GPUs)

- Challenges:
  - Data dependency
    - Instruction *j* is data dependent on instruction *i* if
      - Instruction *i* produces a result that may be used by instruction *j*
      - Instruction *j* is data dependent on instruction *k* and instruction *k* is data dependent on instruction *i*

- Dependent instructions cannot be executed simultaneously

Data Dependence

- Dependencies are a property of programs
- Pipeline organization determines if dependence is detected and if it causes a stall

- Data dependence conveys:
  - Possibility of a hazard
  - Order in which results must be calculated
  - Upper bound on exploitable instruction level parallelism

- Dependencies that flow through memory locations are difficult to detect

Name Dependence

- Two instructions use the same name but no flow of information
  - Not a true data dependence, but is a problem when reordering instructions
  - Antidependence: instruction j writes a register or memory location that instruction i reads
    - Initial ordering (i before j) must be preserved
  - Output dependence: instruction i and instruction j write the same register or memory location
    - Ordering must be preserved
- To resolve, use register renaming techniques

Other Factors

- Data Hazards
  - Read after write (RAW)
  - Write after write (WAW)
  - Write after read (WAR)

- Control Dependence
  - Ordering of instruction i with respect to a branch instruction
    - Instruction control dependent on a branch cannot be moved before the branch so that its execution is no longer controlled by the branch
    - An instruction not control dependent on a branch cannot be moved after the branch so that its execution is controlled by the branch

Examples

- Example 1:

```
    add x1,x2,x3
    beq x4,x0,L
    sub x1,x1,x6
L:  …
    or x7,x1,x8
```

- or instruction dependent on add and sub

- Example 2:

```
    add x1,x2,x3
    beq x12,x0,skip
    sub x4,x5,x6
    add x5,x4,x9
skip:
    or x7,x8,x9
```

- Assume x4 isn't used after skip
  - Possible to move sub before the branch

Compiler Techniques for Exposing ILP

- Pipeline scheduling
  - Separate dependent instruction from the source instruction by the pipeline latency of the source instruction
- Example:

```
for (i=999; i>=0; i=i-1)
  x[i] = x[i] + s;
```

Instruction producing result	Instruction using result	Latency in clock cycles
FP ALU op	Another FP ALU op	3
FP ALU op	Store double	2
Load double	FP ALU op	1
Load double	Store double	0

Pipeline Stalls

```
Loop:   fld      f0,0(x1)
        stall
        fadd.d f4,f0,f2
        stall
        stall
        fsd f4,0(x1)
        addi x1,x1,-8
        bne x1,x2,Loop
```

```
Loop:   fld      f0,0(x1)
        addi x1,x1,-8
        fadd.d f4,f0,f2
        stall
        stall
        fsd f4,0(x1)
        bne x1,x2,Loop
```

Instruction producing result	Instruction using result	Latency in clock cycles
FP ALU op	Another FP ALU op	3
FP ALU op	Store double	2
Load double	FP ALU op	1
Load double	Store double	0

Loop Unrolling

- Loop unrolling
  - Unroll by a factor of 4 (assume # elements is divisible by 4)
  - Eliminate unnecessary instructions

```
Loop:   fld f0,0(x1)
        fadd.d f4,f0,f2
        fsd f4,0(x1) //drop addi & bne
        fld f6,-8(x1)
        fadd.d f8,f6,f2
        fsd f8,-8(x1) //drop addi & bne
        fld f0,-16(x1)
        fadd.d f12,f0,f2
        fsd f12,-16(x1) //drop addi & bne
        fld f14,-24(x1)
        fadd.d f16,f14,f2
        fsd f16,-24(x1)
        addi x1,x1,-32
        bne x1,x2,Loop
```

- note: number of live registers vs. original loop

Loop Unrolling/Pipeline Scheduling

- Pipeline schedule the unrolled loop:

```
Loop:   fld f0,0(x1)
        fld f6,-8(x1)
        fld f8,-16(x1)
        fld f14,-24(x1)
        fadd.d f4,f0,f2
        fadd.d f8,f6,f2
        fadd.d f12,f0,f2
        fadd.d f16,f14,f2
        fsd f4,0(x1)
        fsd f8,-8(x1)
        fsd f12,-16(x1)
        fsd f16,-24(x1)
        addi x1,x1,-32
        bne x1,x2,Loop
```

- 14 cycles
- 3.5 cycles per element

Dynamic Scheduling

- Rearrange order of instructions to reduce stalls while maintaining data flow

- Advantages:
  - Compiler doesn’t need to have knowledge of microarchitecture
  - Handles cases where dependencies are unknown at compile time

- Disadvantage:
  - Substantial increase in hardware complexity
  - Complicates exceptions

Dynamic Scheduling

- Dynamic scheduling implies:
  - Out-of-order execution
  - Out-of-order completion

- Example 1:

```
fdiv.d f0,f2,f4
fadd.d f10,f0,f8
fsub.d f12,f8,f14
```

  - fsub.d is not dependent, issue before fadd.d

Dynamic Scheduling

- Example 2:

  fdiv.d f0,f2,f4

  fmul.d f6,f0,f8

  fadd.d f0,f10,f14

  - fadd.d is not dependent, but the antidependence makes it impossible to issue earlier without register renaming

Register Renaming

- Example 3:

```
fdiv.d f0,f2,f4
fadd.d f6,f0,f8
fsd f6,0(x1)
fsub.d f8,f10,f14
fmul.d f6,f10,f8
```

antidependence

antidependence

  - name dependence with f6

Register Renaming

- Example 3:

```
fdiv.d f0,f2,f4
fadd.d S,f0,f8
fsd S,0(x1)
fsub.d T,f10,f14
fmul.d f6,f10,T
```

- Now only RAW hazards remain, which can be strictly ordered